RAGonomics: Balancing Accuracy and Efficiency in Financial QA

Ali Karim
University of Washington
akarim2@cs.washington.edu

George Lee
University of Washington
george10lee@gmail.com

Abstract

Retrieval-augmented generation (RAG) can be used to efficiently query private and sensitive data. We want to explore how retrieval systems can be used to solve open-ended financial question answering using Large Language Models (LLMs). However, RAG systems require extra token usage and time to perform retrieval, to make up for this, we decided to utilize speculative decoding. We especially want to explore how speculative decoding and context length affect the performance and efficiency tradeoff of RAG-based LLM systems. We are utilizing FinanceBench, an open-source financial question-answering benchmark of 150 questions over SEC filings. Using a fully local pipeline (BGE-small embeddings, a FAISS index, and Qwen2.5-3B-Instruct as the target model with Qwen2.5-0.5B-Instruct as the speculative draft model), we ran every question across four retrieval context budgets (512–4,096 tokens), with reasoning prompting on and off, and with speculative decoding on and off. We found that accuracy rises as retrieved context grows but remains weak across all configurations, that reasoning prompting hurts accuracy rather than helping it, and that speculative decoding produces a slowdown rather than a speedup relative to the baseline. In this paper, we discuss our experimental setup, results, and takeaways about how small local models do on multi-step financial analysis.

1. Introduction

1.1. Introduction & Motivation

Financial analysts are highly paid to search private and sensitive documents, extract tables and numeric data, compute financial metrics, and come to verifiable conclusions. In real world applications, this is an incredibly important task that can influence company spending and the investments in the billions. Large language models (LLMs) are now increasingly tasked to do the same. LLMs now need to search documents like SEC 10-K filings that are long and densely tabular to answer quantitative questions and deter hallucinated answers, after all a wrong answer is worse than no answer. Retrieval-augmented generation (RAG) addresses this problem by forcing the LLM to ground answers in retrieved passages. But this also introduces a huge host of new issues, longer retrieved context can give the model the data it needs, but also inflates prompt length, token usage, memory, and latency. To counter the increased latency, we decided to include speculative decoding techniques to make this system as real world usable as possible. This raises a central question: How speculative decoding and context length affect the performance and efficiency tradeoff of RAG based LLM systems? Since financial documents contain potentially sensitive information, we wanted to focus on locally run open-source LLMs as opposed to third-party hosted models.

1.2. Background

Across this paper, we refer to a few core concepts that require additional explanation.

1.2.1. Retrieval-Augmented Generation

Retrieval-Augmented Generation helps ground LLM to real world truths by providing external evidence to be used at inference time, rather than just relying on its own training. A corpus of documents is chunked and then converted into a dense vector embedding. These embeddings are then stored in a vector database. At query time, the user's question is embedded with the same embedding model and the cosine similarity between the prompt and related chunks is computed to retrieve some top-k chunks that are then provided to the LLM's prompt as additional context, allowing the model to answer from document-grounded evidence.[8]

1.2.2. Speculative Decoding

Speculative Decoding is a method that helps improve model inference speed. By leveraging a smaller faster "draft" model to propose multiple tokens at a time, a larger "target" model then verifies the tokens generated in a single forward pass. Using a greedy verification, the target model validates every accepted token, so the output is theoretically lossless. For large target models, there is usually a 2-3x speed up, but that depends on the draft model's acceptance rate. If the two models disagree continuously, the

generation time can slow down instead of speed up. Also, if the target model is too small, speculative decoding gives little to no speedup since the target model is already very fast compared to the baseline. Both the target and draft model must come from the same model family, have compatible tokenizers, and the same vocabulary size.[7]

1.3. Related Works

Our research's goal is to study the intersection of context length, RAG, and speculative decoding, and determine the accuracy-efficiency trade off. In 2024, a group at DataBricks published Long Context RAG Performance of Large Language Models. They looked at how retrieved context length affects RAG performance and investigated using the Qwen2-72B-instruct model and evaluated it on the FinanceBench dataset along with other models and datasets.[6] Unlike our experiment, they did not use reasoning variation of their models. Another paper we looked at was Fast Inference from Transformers via Speculative Decoding. This paper discusses how speculative decoding actually works; a smaller draft model proposes several tokens at a time and the larger model, and the larger model verifies in a forward pass. Under a greedy verification, the output is identical to standard decoding, and the paper reports a 2-3x speed up for larger target models.[7] Another application of speculative techniques is in the paper, Improving speculative retrieval-augmented generation via verifier scoring. The paper discusses Speculative RAG, the idea of using the speculative process for the retrieval of documents. The paper discusses splitting the RAG process into a two stage process like speculative decoding.[2] These papers show the previous work and important applications of what we intend to explore, context budgeting and speculative execution can improve the accuracy-efficiency frontier and help these systems perform better and more cost-efficiently.

2. Methodology

2.1. Dataset

We use the open-source version of FinanceBench [5], a benchmark for open-ended financial question answering over SEC filings. The open-source release contains 150 questions about publicly traded companies and provides 361 SEC filing PDFs [1]. Each question includes an annotated "gold" answer and supporting evidence from the filings. The benchmark covers multiple forms of financial reasoning, including comparison, average, lookup and extraction, percent and margin, ratio, change and growth, derived financial metrics, and other question types.

2.2. Models

Our generation setup used Qwen2.5-3B-Instruct[10] as the target model and, Qwen2.5-0.5B-Instruct as the draft model for speculative decoding. We considered several larger and newer alternatives, including Qwen2.5-9B and newer Qwen releases like Qwen3.5, but found them less appropriate for our constraints. Larger models suffered from Out-Of-Memory(OOM) errors, and used an incompatible tokenizer when compared to Qwen2.5-0.5B, leading to unacceptable low draft-acceptance rates. Newer models provided their own challenges, with different speculative decoding implementations being difficult for us to measure and validate.

For chunk and question embedding, we used BAAI/bge-small-en-v1.5[9], a compact open-source embedding model suitable for our local use. All models were run on a NVIDIA A100, chosen due to its balance of ubiquitous industry use, and relatively low cost.

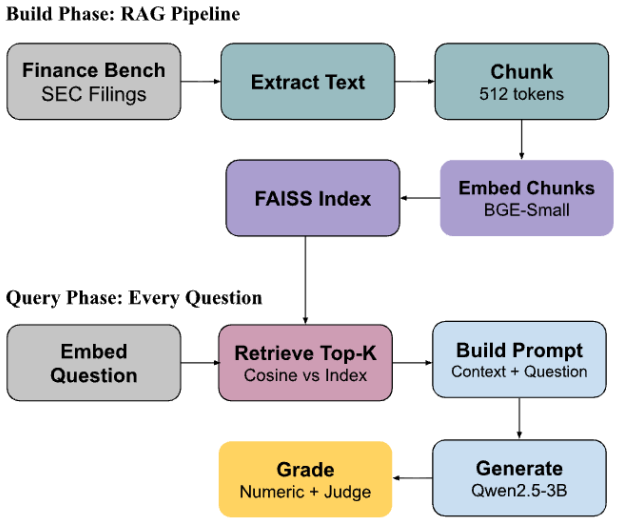

Figure 1. Overview of the local RAG pipeline. FinanceBench SEC filings are converted to text, split into overlapping chunks, embedded with BGE, indexed with FAISS, retrieved by cosine similarity, and inserted into the prompt for Qwen2.5 generation.

2.3. RAG Pipeline

For each document in FinanceBench, we extracted the raw text from each page using PyMuPDF, a lightweight python library for PDF analysis. The raw text is then broken into 512 token sized chunks, with a 64 token overlap to hopefully preserve information across chunk boundaries when relevant ideas span across multiple chunks. We tokenize the chunks using Qwen's tokenizer to keep measurement variables consistent, and then embed the chunks using BAAI/bge-smal [9] to produce dense semantic embedding vectors fit for contextual retrieval. These vectors were then L2-normalized, allowing for inner-product search using cosine similarity. We stored these embeddings, along with a metadata link to their textual context in Facebook AI Sim-

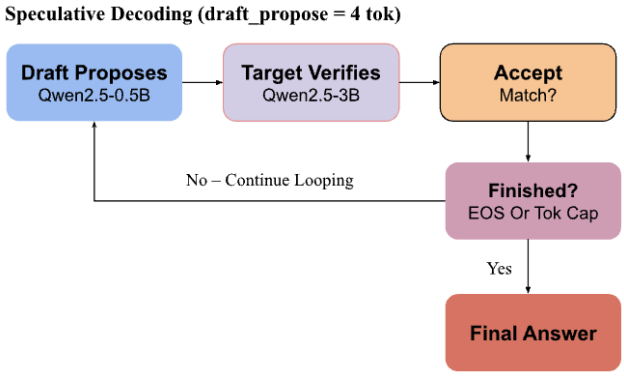

Figure 2. Speculative decoding setup. The smaller draft model proposes four future tokens, and the larger target model verifies those proposals.

ilarity Search(FAISS) [3], a light weight vector database meant for efficient searching of dense vectors. This process yielded a total of 103,909 chunks. At retrieval time, the question is embedded along with a unique retrieval instruction prefix as suggested by BGE retrieval convention. [4] We prepend a query instruction prefix to the question before embedding it, while document chunks are embedded directly. The retriever then selects the top-k chunks by cosine similarity. These retrieved chunks are converted back into text and inserted before the question in a fixed prompt template.

2.4. Speculative Decoding Generation

The speculative decoding generation is greedy (no sampling). We used a fixed cap of 1024 new tokens held constant for all token generations. This allows the reasoning-on model configurations to generate answers without getting cut off. The draft model proposes four tokens at a time and then the target verifies them. Because we are using greedy decoding, the output should remain theoretically lossless. We also time the token generation for each variation of our model to record per-answer latency, generated-token count, and tokens-per-second. We then compute speculative decoding speedup as

$$\text{Speedup} = \frac{T_{\text{base}}}{T_{\text{spec}}},$$

where T_{base} is the baseline generation latency with speculative decoding disabled, and T_{spec} is the generation latency with speculative decoding enabled.

2.5. Grading

All gold answers are hand annotated, so they can be formatted in several ways. So, once the model generates a financial answer, the answer is first checked for numerical correctness. If the gold answer contains a number, we check to see if the model's answer and the gold answer are within 1% of each other. Answers have to be normalized to the thousand, million, billion or percentages based on the question.

If the gold answer is non-numeric, the fall back is a Judge LLM, we used the Qwen2.5-7B model, that returns a binary CORRECT/INCORRECT decision. After obtaining our results, we manually reviewed all model answers compared to the gold answer.

3. Experimental Setup

In our experiment, we run a full grid search over retrieved context length, reasoning prompting, and speculative decoding. We hold the models, dataset, GPU, prompt template, RAG pipeline, and maximum generation length constant across all runs.

3.1. Experimental Grid

Controlled variables. The model, dataset, GPU, prompt template, RAG pipeline, and maximum generation length are held constant across all experimental runs.

Table 1. Controlled variables held constant across all experimental runs.

Variable	Value
Target model	Qwen2.5-3B-Instruct
Draft model	Qwen2.5-0.5B-Instruct
Judge model	Qwen2.5-7B-Instruct
Dataset	FinanceBench - Open Source Ver.
GPU	NVIDIA A100
Prompt template	Fixed across all runs
Max new tokens	1024
RAG pipeline	Fixed across all runs

Independent variables. We search across retrieved context length, reasoning prompting, and speculative decoding.

Table 2. Independent variables varied in the experiment.

Variable	Values
Retrieved context	512, 1024, 2048, 4096 tokens
Reasoning prompting	On, Off
Speculative decoding	On, Off

The retrieved context settings correspond to $k \in \{1, 2, 4, 8\}$ retrieved chunks, where each chunk contains approximately 512 tokens.

3.2. Evaluation Metrics

We evaluate each run using four metrics: accuracy, per-answer latency, generated tokens per second, and specula-

tive decoding speedup. Accuracy is measured against the FinanceBench gold answer. Per-answer latency is measured as the time required to generate a complete answer, excluding retrieval and prompt construction. Tokens per second is computed as the number of generated tokens divided by generation latency. Speculative decoding speedup is computed using the formula as defined in Sec. 2.4.

3.3. Experiment Size

The complete grid contains

$$150 \times 4 \times 2 \times 2 = 2400$$

total generations, corresponding to 150 questions, 4 retrieved-context lengths, 2 reasoning settings, and 2 decoding modes. All runs completed with zero out-of-memory or runtime errors.

4. Results

Table 3 reports the accuracy, throughput (tokens-per-second), and latency for all sixteen model configurations. There are three main patterns that emerge from this table; accuracy increases as retrieved context increases, reasoning-on models have lower accuracy than reasoning-off models, and speculative-on models have lower token-per-second generation compared to their baseline.

Table 3. All model configurations ranked by their retrieved-tokens (full run, 150 questions per configuration).

Retrieved Tokens	Reasoning	Mode	Accuracy	Tok/s	Latency
512	Off	Spec Off	16.7%	22.98	6.05
512	Off	Spec On	18.0%	19.26	6.90
512	On	Spec Off	12.7%	23.03	7.78
512	On	Spec On	12.7%	20.08	9.09
1024	Off	Spec Off	18.0%	22.90	6.95
1024	Off	Spec On	19.3%	19.82	8.14
1024	On	Spec Off	16.0%	22.95	9.08
1024	On	Spec On	18.7%	20.84	9.89
2048	Off	Spec Off	22.0%	22.69	8.81
2048	Off	Spec On	24.0%	20.58	9.38
2048	On	Spec Off	20.7%	22.80	9.13
2048	On	Spec On	21.3%	20.82	9.74
4096	Off	Spec Off	25.3%	22.24	8.77
4096	Off	Spec On	25.3%	20.06	9.53
4096	On	Spec Off	21.3%	22.34	8.76
4096	On	Spec On	22.7%	20.58	9.73

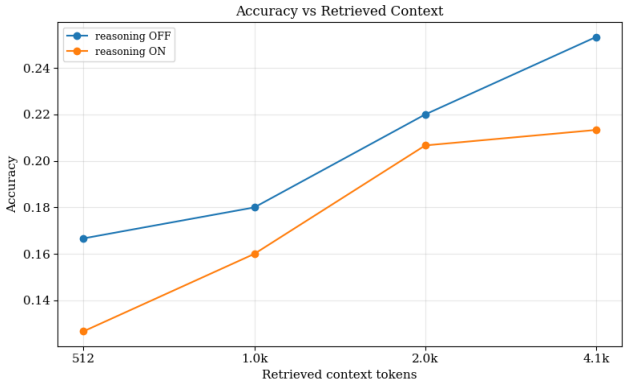

Figure 3. Accuracy vs retrieved context. Accuracy of both reasoning-on and reasoning-off models plotted against the amount of retrieved context measured in tokens. Both models show an increase in accuracy as retrieval context increases. Accuracy climbs from 16.7% at 512 retrieved tokens to 25.3% at 4,096 with reasoning-off, and from 12.7% to 21.3% with reasoning-on (Table 3).

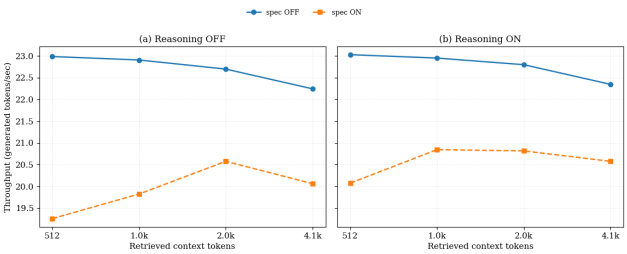

Figure 4. Throughput (generated tokens/sec) vs retrieved context length.

Speculative decoding on models is slower than the baseline model at every context length. Our experiment shows that there is no noticeable benefit to using speculative decoding to increase token generation speed. Reasoning-on and reasoning-off models generate a similar number of tokens per second.

Table 4. Speculative decoding speedup for each model configuration.

Retrieved Tokens	Reasoning	Speedup
512	Off	0.87
512	On	0.88
1024	Off	0.85
1024	On	0.91
2048	Off	0.94
2048	On	0.96
4096	Off	0.91
4096	On	0.92

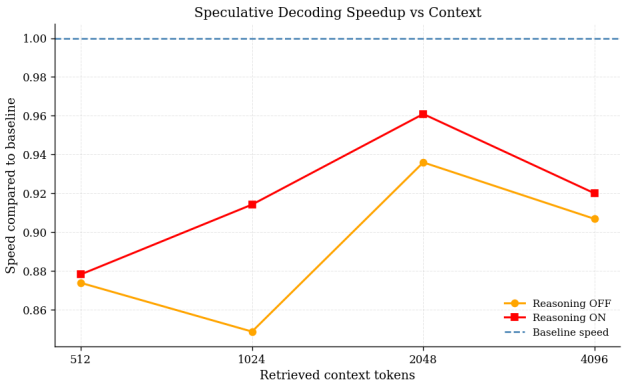

Figure 5. Speculative decoding speedup compared to baseline vs retrieved context.

Table 4 and Fig. 5 show speculative decoding results compared to their baseline models. Performance was closest to baseline at the 2048 context tokens, suffering only a speed reduction of 4% in the reasoning-on model and 6% in the reasoning-off model. The largest slowdown occurred at reasoning-off with 1024 retrieved tokens, where speculative decoding was roughly 15% slower than baseline.

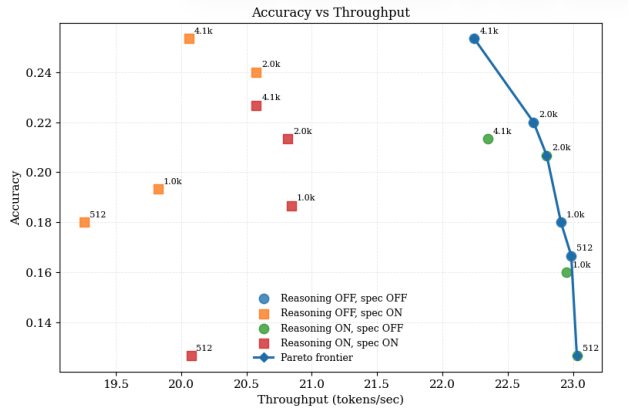

Figure 6. Accuracy vs throughput. Pareto Curve of all model configurations.

Our results show that the best configuration of models is reasoning-off and speculative decoding off. At every retrieval context length, these models have the best accuracy-efficiency tradeoff.

Table 5. Accuracy and speed by question type. The table shows on average how the combined model-set achieved on each question type. Models performed the best on simple questions like comparisons, averages, and lookups, and were challenged by multi-step questions like change/growth and derived financial metric questions. In this case speedup is calculated per question and then averaged by question type.

Question Type	n	Acc. Base	Acc. Spec	Speedup	Tok/s Base	Tok/s Spec
Comparison	2	50.0%	50.0%	1.04	22.91	22.24
Average	9	29.2%	31.9%	0.93	22.85	21.02
Lookup / extraction	19	26.3%	24.3%	0.91	22.70	20.22
Percent / margin	42	20.2%	21.1%	0.94	22.81	20.61
Ratio	19	17.8%	20.4%	0.90	22.65	20.18
Change / growth	21	10.1%	11.3%	0.88	22.76	20.30
Derived financial metric	2	0.0%	0.0%	0.95	22.91	22.01
Other	36	16.7%	18.8%	0.86	22.68	19.47

5. Discussion

5.1. Accuracy

Our results show that the accuracy across all model variations is very low, with the best case being 25.3% and the worst case being 12.7% (as shown in Fig. 3). The accuracy also varies greatly by question type, performing better on simple questions like lookup and comparison, but fails on questions that require multi-step computation (as shown in (Table 5). The small sizes of our draft and target models are the primary driver of weak performance. The model can consistently find the correct passage of text, but then makes mathematical or logical mistakes when calculating the answer to the given question. Another thing of note is that unlike what we initially thought, enabling reasoning reduces accuracy rather than improving it. We believe that this is due to the 3B model's thinking chain introducing errors in the intermediate steps or increasing the chances of hallucination. A point of interest was that the Judge LLM graded very strictly and occasionally misgraded borderline cases, so manual review was required to validate results.

5.2. Speculative Decoding

In our experiment, speculative decoding did not improve token generation speed for any of our model configurations compared to their baseline (see Table 4). There are two possible explanations for this. First, the acceptance rate of the 0.5B draft model is limited, the 0.5B and the 3B models often disagree on the next token, so the proposed tokens are rejected and the extra tokens drafted during the forward pass are wasted. The second reason is that the target model is too small. Since both models are relatively small and close in size, there is little latency difference between both models generating answers. So, when adding in the speculative decoding overhead, latency increases. The latency of the overhead is greater than the time savings of speculative decoding, so the system slows down rather than speeds up. Another thing to note is that although we used greedy speculative decoding, the output was not lossless, the output

matches approximately 55% of the time. This could be for several reasons, but it is most likely due to a configuration mismatches between models.

5.3. Retrieved Context

Our results show that increasing the amount of retrieved context did improve accuracy. Additional chunks bringing in new and relevant information can help the model reason. As the retrieved context increases, accuracy improves while throughput falls, which is exactly the accuracy-efficiency relationship we were looking to explore (see Fig. 6). As shown in the plotted pareto curve, with a fixed decoding mode, the increase in retrieval context (and the resulting increase in latency) is worth the corresponding increase in accuracy.

5.4. Limitations

Our study faced a few limitations:

5.4.1. Model Choice

Our choice of models was constrained to potential local LLMs capable of running a full experimental setups with exposed speculative decoding measurements. Newer versions of Qwen such as Qwen 3.5 have the potential to perform significantly better, but the evaluation architecture built around them has yet to catch up with SOTA models and would be too much overhead for the goals of our experiment. Additionally, while we prompted Qwen2.5 with a basic chain-of-thought template, newer Qwen models have been trained specifically to support explicit reasoning behavior through dedicated reasoning and non-reasoning modes, which could have improved reasoning performance on our dataset.

5.4.2. Retrieval-Augmented Generation

While our RAG pipeline was very effective, it had some limitations. The pipeline only extracted text layers from each PDF, so no tables or figures were captured. The pipeline also chunked text by 512 tokens, so information across chunks and documents could be lost and harder to retrieve. We attempted to mitigate this by overlapping chunks with 64 tokens.

6. Conclusion

From our experiment, we can draw several important conclusions. We discovered that as retrieved context grows, accuracy improves while throughput (tokens-per-second) falls, highlighting the accuracy-speed trade-off, with diminishing accuracy returns at the largest context sizes. Using the 0.5B draft model and the 3B target model, speculative decoding does not yield any speedup. Enabled reasoning models actually hurt model performance rather than help it. These models perform better on simple questions like comparison, averages, and lookups, but are challenged by multi-step computation question types like derived financial metrics.

7. Future Work

There are several areas of research that we are interested in next. First, we would like to explore Speculative RAG, applying speculative techniques to retrieval systems, to investigate the efficiency of the retrieval process. We are also excited about using multimodal models to perform financial related tasks. We want to examine how accuracy can be improved when multimodal models are used to read tables and charts that text only models cannot. We would also like to rerun the same experiment, but with more compute, larger context retrieval lengths, and stronger models. Our goal would be to determine where speculative decoding's savings outweigh its overhead, and further explore the relationship between accuracy and context retrieval.

References

[1] Patronus AI. Financebench: Sec filings pdfs. https://github.com/patronus-ai/financebench, 2023. 2

[2] Shixin Chen. Improving speculative retrieval-augmented generation via verifier scoring. Master's thesis, University of Illinois Urbana-Champaign, 2025. 2

[3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazare, Maria Lomeli, Lucas Hosseini, and Herve Jegou. The faiss library, 2024. 3

[4] Hanhainebula and Shitao Xiao. Flagembedding: Open-source embedding toolkit. https://pypi.org/project/FlagEmbedding/, 2023. 3

[5] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. Financebench: A new benchmark for financial question answering, 2023. 2

[6] Quinn Leng, Jacob Portes, Sam Havens, Matei Zaharia, and Michael Carbin. Long context rag performance of large language models, 2024. 2

[7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. 2

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2020. 1

[9] Shitao Xiao, Zheng Liu, Jianlv Chen, and Peitian Zhang. bge-small-en-v1.5: A compact english text embedding model. https://huggingface.co/BAAI/bge-small-en-v1.5, 2023. 2

[10] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, and et al. Qwen2.5 technical report, 2024. 2